

5월 22일 수업용 메모 요약본

[Day 9] 디지털 보안/암호화 이론 & Docker 가상화 실습 총정리

 **Notion 업로드 팁:** 본 문서는 노션(Notion)의 마크다운 변환에 최적화되어 있습니다. 전체 선택(`Ctrl + A` → `Ctrl + C`) 후 노션 빈 페이지에 붙여넣기(`Ctrl + V`) 하시면 깔끔한 토글(Toggle), 콜아웃(Callout), 테이블(Table) 형태로 자동 렌더링됩니다.

1교시. 암호 알고리즘의 3대 패러다임

리눅스 및 네트워크 보안에서 사용되는 암호 알고리즘은 용도와 키의 사용 여부에 따라 크게 세 가지 범주로 분류됩니다.

구분	주요 알고리즘	특징	키(Key) 사용 여부	대표 실무 명령어
① 해시 (Hash)	SHA-256, SHA3	단방향(복호화 불가), 임의의 입력을 고정 길이로 요약	X (키 없음)	<code>sha256sum <파일></code>
② 대칭키 (Symmetric)	AES-256-CBC	암호화와 복호화에 동일한 1개의 비밀키 사용	O (1개 공유키)	<code>openssl enc -aes-256-cbc</code> ...
③ 비대칭키 (Asymmetric)	RSA, ECC	공개키(Public)와 개인키(Private) 쌍 사용	O (2개 쌍)	<code>openssl genrsa</code> ...

2교시. 해시 함수와 무결성 검증 실습

1) 해시(Hash) 함수의 특징

- **단방향성 (One-Way):** 평문을 암호문으로 바꾸는 것은 쉽지만, 암호문을 평문으로 되돌리는 역산은 수학적으로 불가능합니다.
- **눈사태 효과 (Avalanche Effect):** 입력값의 아주 미세한 변화(예: 대소문자 전환, 마침표 추가)가 완전히 무작위의 다른 결과값을 생성해 냅니다.

```
# 실제 실습 로그 분석: 소문자 'h'와 대문자 'H'의 차이
kopo20@Ubuntu:~$ echo "hello" | sha256sum
5891b5b522d5df086d0ff0b110fbd9d21bb4fc7163af34d08286a2e846f6be03 -

kopo20@Ubuntu:~$ echo "Hello" | sha256sum
66a045b452102c59d840ec097d59d9467e13a3f34f6494e539ffd32c1bb35f18 -
```

2) 실무 시나리오: 파일 무결성(Integrity) 검증

네트워크를 통해 다운로드한 파일이 해커에 의해 위변조되었는지 검증할 때 사용하는 표준 프로세스입니다.

1단계: 원본 파일 작성 및 원본 해시 값 저장 (**.sha256**)

```
echo "Linux is fun." > readme.txt
sha256sum readme.txt > readme.sha256

# 해시맵 정보 확인 (해시값과 대상 파일명이 공백으로 매핑됨)
cat readme.sha256
# 출력: 7135f06d61d96460221021f7f51e1d572157c7dd77e4e24e3a4cb8d035dd14c1  readme.txt
```

2단계: 누군가 파일을 변조함 (소문자 'fun' ➡ 대문자 'FUN')

```
echo "Linux is FUN." > readme.txt
```

3단계: **sha256sum -c** 검증 도구 실행

```
sha256sum -c readme.sha256
# 출력 결과:
# readme.txt: FAILED
# sha256sum: WARNING: 1 computed checksum did NOT match
```

- **결과 해독:** 원본 해시맵 정보(**7135f06...**)와 변조된 파일의 실시간 해시값이 일치하지 않으므로 시스템이 **FAILED** 를 발생시켜 위변조를 정확히 감지해 냅니다.

3) 솔팅(Salting)의 직접적인 검증

단순한 비밀번호(1234)는 해커들의 '레인보우 테이블(미리 계산된 해시맵 사전)'에 의해 쉽게 무력화되므로 임의의 salt 데이터를 결합해 해싱해야 합니다.

```
# 기본 해싱
echo -n "1234" | sha256sum
# 결과: 03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4 -

# 서로 다른 소금값(ab12, cd34)을 버무렸을 때 결과 지문이 완벽하게 분리됨
echo -n "ab12 1234" | sha256sum # 결과: 9172eec...
echo -n "cd34 1234" | sha256sum # 결과: 3f668d2...
```

3교시. OpenSSL 대칭키 & 비대칭키 암호화 실습

1) AES-256-CBC 대칭키 암호화 실습

대칭키 방식은 단 하나의 공유 비밀키를 사용하여 매우 고속으로 통신 데이터를 잠그고 푸는 실무 표준 방식입니다.

```
# 평문 생성
echo "내 통장번호는..." > secret.txt

# [암호화]: -pbkdf2(키 유도 함수) 옵션과 함께 AES-256-CBC로 암호화 진행 (패스워드 지정)
openssl enc -aes-256-cbc -salt -pbkdf2 -in secret.txt -out secret.enc

# 결과 확인 (바이너리로 깨진 외계 문자 출력 확인)
cat secret.enc

# [복호화]: 대칭 암호키 패스워드를 입력하여 평문 복구
openssl enc -d -aes-256-cbc -pbkdf2 -in secret.enc -out out.txt
```

2) RSA 2048 비대칭키(공개키/개인키) 실습

비대칭키 방식은 누구나 볼 수 있는 공개키(Public Key)와 주인만 비밀리에 보관하는 개인키(Private Key) 쌍을 이용해 키 유출 문제를 차단합니다.

Step 1. 2048비트 RSA 개인키 생성

```
openssl genrsa -out private-key.pem 2048
```

Step 2. 개인키로부터 한 쌍인 공개키 파일만 안전하게 추출

```
openssl rsa -in private-key.pem -pubout -out public-key.pem
```

Step 3. 공개키를 이용한 암호화 (수신자의 우체통 투입구)

전 세계 유저들에게 공개된 `public-key.pem` 으로 데이터를 숨깁니다.

```
echo "steven0123:abcd1234" > plain.txt
openssl pkeyutl -encrypt -pubin -inkey public-key.pem -in plain.txt -out cipher.bin
```

Step 4. 개인키를 이용한 복호화 (수신자 고유의 집배원 열쇠)

서버 깊숙이 보관된 `private-key.pem` 소유자만이 이 데이터를 해독할 수 있습니다.

```
openssl pkeyutl -decrypt -inkey private-key.pem -in cipher.bin -out decrypted.txt
cat decrypted.txt
# 결과: steven0123:abcd1234
```

3) 디지털 서명 (Digital Signature) 실습

디지털 서명은 "작성자의 신원 보증(인증)"과 "데이터의 위변조 방지(무결성)"를 동시에 확보하기 위해 비대칭키 원리를 뒤집어 사용합니다.

- **서명(Sign):** 원본 데이터를 요약(Hash)한 뒤, 작성자의 개인키(Private)로 암호화하여 서명본(`msg.sig`)을 만듭니다.
- **검증(Verify):** 누구나 획득 가능한 작성자의 공개키(Public)로 서명을 복호화하여 일치 여부를 검증합니다.

```
# 1. 원본 파일 생성
echo "release v1.0" > msg.txt

# 2. 내 개인키(my.key)를 이용해 서명 파일(msg.sig) 생성
openssl dgst -sha256 -sign my.key -out msg.sig msg.txt

# 3. 내 공개키(my.pub)를 대조하여 서명이 무결한지 검증
openssl dgst -sha256 -verify my.pub -signature msg.sig msg.txt
# 출력: Verified OK (정상 서명 완료)
```

위변조 챌린지 및 실패 감지

만약 악의적인 인물이 서명 파일은 그대로 둔 채 원본 파일만 교묘하게 수정한다면?

```
echo "release v2.0" > msg.txt # 임의 조작
openssl dgst -sha256 -verify my.pub -signature msg.sig msg.txt
# 결과: Verification failure (위변조 완벽 차단!)
```

4교시. 사설 인증기관(CA) 구축 및 SSL/TLS 인증서 발급

실제 보안 실무 환경에서 HTTPS 통신망을 개설하기 위해 사용하는 **Chain of Trust (신뢰 사슬)** 구축의 풀-사이클 실습입니다.

```
[ Root CA 자가 생성 ] → [ Server CSR 요청서 작성 ] → [ Root CA 개인키로 승인/서명 ] → [ 인증서 체인 검증 완료 ]
rootCA.key & .pem          server.csr 생성          certificate-public.pem 발급          verify OK
```

Step 1. 최상위 루트 인증기관(Root CA) 환경 구축

사설 CA의 루트 개인키와 이를 증명할 루트 자체 서명 인증서(Self-Signed)를 만듭니다.

```
# CA 자체 4096비트 개인키 생성
openssl genrsa -out rootCA.key 4096

# 10년(3650일) 동안 유효한 루트 인증서 발급 (CN을 KopoRootCA로 인가)
openssl req -x509 -new -key rootCA.key -days 3650 -out rootCA.pem -subj "/CN=KopoRootCA"
```

Step 2. 서버용 인증서 신청서(CSR) 만들기

도메인(stevenjlee.net)의 웹 서비스를 구동하기 위해 서버 전용 개인키와 기관 승인용 인증서 요청파일(CSR)을 빌드합니다.

```
# 서버 전용 2048비트 개인키 생성
openssl genrsa -out private-key.pem 2048

# 도메인 정보와 개인키를 연계하여 CSR 서류 빌드
openssl req -new -key private-key.pem -out server.csr -subj "/CN=stevenjlee.net"
```

Step 3. 루트 CA의 비밀키로 서버 인증서 서명 및 발급

최상위 인증기관의 개인키(`rootCA.key`)를 사용해 서버의 요청서(`server.csr`)에 인장을 찍어 공식 인증서(`certificate-public.pem`)를 발행합니다.

```
openssl x509 -req -in server.csr \
  -CA rootCA.pem -CAkey rootCA.key -CAcreateserial \
  -out certificate-public.pem -days 365
```

Step 4. 발급된 SSL/TLS 인증서 신뢰 사슬(Chain) 검증

```
openssl verify -CAfile rootCA.pem certificate-public.pem
# 출력 결과: certificate-public.pem: OK (인증 성공!)
```

Step 5. 발급 완료된 인증서 상세 속성(Metadata) 조회

```
openssl x509 -in certificate-public.pem -noout -subject -issuer -dates -fingerprint -sha256
# 출력값 해독:
# subject=CN = stevenjlee.net      (소유주: 내 서버 도메인)
# issuer=CN = KopoRootCA          (발급자: 신뢰받는 내 루트 CA)
# notBefore=May 22 ... 2026 GMT   (유효 시간 검증)
# notAfter=May 22 ... 2027 GMT
# sha256 Fingerprint=AB:42:...    (고유 디지털 지문 확인)
```

5교시. Docker 가상화 기초 및 물리적 공유 개념

하드웨어를 통째로 모방하는 하이퍼바이저 방식(VM)과 달리, **Docker 컨테이너 가상화**는 호스트의 커널(Kernel)을 공유하면서 프로세스 단에서 독립된 사용자 공간(User Space)을 격리시킵니다.

1) 격리 메커니즘 직접 검증하기 (실습 분석)

우분투 가상머신(호스트) 위에서 `ubuntu` 도커 컨테이너를 가동하고 시스템 사양을 대조해 봅니다.

```
# 1. 우분투 컨테이너를 띄우고 내부 Bash 셸 진입
kopo20@Ubuntu:/home$ docker run -it --name myubu ubuntu bash
```

```
# 2. 컨테이너 내부에서 커널 버전 확인
root@bde7cb8c01fb:/# uname -r
6.8.0-117-generic # -> 호스트 우분투 머신과 '완벽히 동일한' 커널을 사용하고 있음!

# 3. 컨테이너 내부의 배포판 OS 정보 확인
root@bde7cb8c01fb:/# cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04 LTS" # -> 가상 환경 내부에 완벽히 격리된 파일시스템 탑재!
```

- **결론:** 컨테이너 내부 OS 구조는 **Ubuntu 26.04** 로 완전히 격리되어 보이지만, 하부 커널 구조(**6.8.0**)는 윈도우/호스트의 Linux 커널을 그대로 공유하여 연산 속도를 극대화합니다.

2) 🧹 컨테이너 1회성 실행 및 자동 삭제 (**-rm**)

가볍게 명령어를 한 번만 테스트하고, 찌꺼기 컨테이너 파일을 로컬에 남겨두고 싶지 않을 때 사용하는 실무 표준 태그입니다.

```
kopo20@Ubuntu:/home$ docker run -it --rm ubuntu bash -c "echo hi from inside"
hi from inside
```

- 이 명령이 종료된 후 **docker ps -a** 를 입력하면 컨테이너 기록이 깨끗하게 자동 삭제되어 디스크 공간 낭비를 방지합니다.

🔌 6교시. Docker 포트 포워딩 및 실무 문제 제어

도커 컨테이너 내부 포트(예: Nginx의 80번 포트)를 외부 사용자가 접속할 수 있도록 가상머신 호스트의 외부 포트(예: 8080)와 가교를 연결하는 개념이 포트 포워딩(**-p**)입니다.

🔄 포트 바인딩 핑퐁 메커니즘

```
[ 외부 클라이언트 ] -> [ 호스트 Windows / VM (Port 8080) ] -> [ Docker Container (Port 80) ]
```

```
# 호스트의 8080 포트로 유입되는 트래픽을 컨테이너 80 포트로 연결 및 백그라운드(-d) 구동
docker run -d --name web1 -p 8080:80 nginx
```

🏗️ 7교시. [실무 실전] 나만의 웹 서버 구축 및 배포 (serverstatus.html)

도커 빌드(`docker build`)는 단순히 미리 준비된 템플릿을 실행하는 것이 아니라, 사용자가 직접 정의한 리소스(`serverstatus.html` 및 `ChartNew` 라이브러리 폴더)를 활용해 커스텀 서비스를 패키징하고 가동하는 전 과정입니다.

```
[ 1단계: 실습 리소스 검수 ] -> [ 2단계: Dockerfile 제작 ] -> [ 3단계: 커스텀 이미지 빌드 ] -> [ 4단계: 컨테이너 배포 및 확인 ]
mysite 폴더 및 리소스 확인      Vim으로 COPY 지시 명시      docker build -t mysite .
http://localhost:8080/
```

1단계: 실습 리소스 확인 및 디렉토리 위치 선정

현재 내 가상머신의 `~/mysite` 디렉토리에 교수님이 배포해 주신 실시간 대시보드 리소스들이 올바르게 담겨 있는지 먼저 점검합니다.

```
cd ~/mysite
ls -l
# 출력 결과 확인:
# index.html
# serverstatus.html
# ChartNew/ (폴더 내에 ChartNew.js가 존재해야 함)
```

2단계: Dockerfile 작성 (Task 2)

`serverstatus.html` 은 차트를 렌더링하기 위해 하위 폴더의 `ChartNew.js` 라이브러리를 동적으로 로드합니다. 따라서 빌드 과정에서 **HTML 파일과 폴더를 통째로 컨테이너 내부 Nginx 기본 웹 경로에 주입**하도록 지시해야 합니다.

```
vi Dockerfile
```

• Dockerfile 내부 기입 내용:

```
FROM nginx:latest

# 1. 수동 제작한 index.html 복사
COPY index.html /usr/share/nginx/html/index.html

# 2. 시스템 계측용 대시보드 파일(serverstatus.html) 복사
COPY serverstatus.html /usr/share/nginx/html/serverstatus.html
```

```
# 3. ChartNew 라이브러리 폴더를 컨테이너 내부 하위 디렉토리로 통째로 복사
COPY ChartNew/ /usr/share/nginx/html/ChartNew/
```

(※ Vim 에디터에서 **Esc** 누르고 **:wq** 로 저장하여 파일을 닫아줍니다.)

3단계: 나만의 도커 이미지 빌드하기 (Task 3)

작성된 도면(**Dockerfile**)을 빌드하여 **mysite** 라는 전용 컨테이너 이미지를 컴파일해 냅니다.

```
# 끝부분에 공백 한 칸 띄우고 온점(.) 필수 입력! (현재 디렉토리를 Context로 지정)
docker build -t mysite .
```

4단계: 포트 충돌 우회하여 컨테이너 실행하기 (Task 4)

기존에 **8080** 포트를 선점하고 있던 타 프로세스나 컨테이너를 완전히 Shutdown 시킨 상태에서 새로 빌드한 **mysite** 컨테이너를 가동합니다.

```
# 호스트의 8080 포트를 점유하여 Nginx 내부의 80 웹 서버 포트와 연결
docker run -d --name mysite-web -p 8080:80 mysite
```

5단계: 웹 브라우저에서 최종 배포 화면 계측 (Task 5)

1. 윈도우의 Chrome 등 브라우저 주소창을 엽니다.
2. 아래 주소에 본인의 가상머신 호스트 전용 IP(**192.168.56.X**)를 넣어 접속합니다.
 - 기본 페이지: **http://192.168.56.101:8080** (Built into image 문구 송출 확인)
 - 시스템 대시보드 페이지: **http://192.168.56.101:8080/serverstatus.html**
3. 정상적으로 **CPU, Memory, Disk Status** 그래프가 동적 바인딩되어 출력되는지 검증합니다.

8교시. 실무 및 보안/Docker 실습 단골 오류 & 해결 가이드

[Toggle] 에러 1: **/home** 디렉토리 아래 파일 쓰기 실패 (**Permission denied**)

- 상태: **kopo20@ubuntu:/home\$ echo "hello" > readme.txt** 입력 시 권한 거부 오류 발생.

- **원인:** `/home` 경로는 각 사용자들의 전용 폴더들이 배치되는 최상위 시스템 디렉토리로, root 관리자가 아닌 일반 사용자(`kopo20`)는 쓰기 권한이 막혀 있기 때문입니다.
- **해결 방법:** 본인의 홈 디렉토리 하위의 개인 작업 공간(`~/workspaces/day9`)으로 이동한 뒤 파일 생성을 진행해야 합니다.

```
cd ~/workspaces/day9
```

▼ [Toggle] 에러 2: 디스크 경로 이동 실패 (`Not a directory`)

- **상태:** `cd /home/kopo20/workspaces/day8/day9` 이동 시 디렉토리가 아니라는 경고 발생.
- **원인:** `day8` 폴더 아래에 `day9` 을 만든 것이 아니라, `workspaces` 폴더 하위에 동등하게 `day9` 을 생성해 두었기 때문에 잘못된 경로를 참조하여 발생합니다.
- **해결 방법:** 올바른 계정 홈 하위 독립 디렉토리 경로로 이동합니다.

```
cd /home/kopo20/workspaces/day9
```

▼ [Toggle] 에러 3: OpenSSL Symmetric 복호화 실패 (`bad decrypt` 에러)

- **상태:** 대칭 암호 해독 명령 시 `bad decrypt` 및 에러 발생.
- **원인:** 암호화할 때 지정했던 대칭 암호키(패스워드) 값과 복호화할 때 입력한 비밀번호 텍스트가 일치하지 않아 수학적 패딩 블록의 일치성이 붕괴하여 발생하는 전형적인 보안 에러입니다.
- **해결 방법:** 암호화에 사용했던 패스워드를 정확히 기록해 둔 뒤, 복호화 시 동일한 암호키 문자열로 가동해 주어야 합니다.

▼ [Toggle] 에러 4: `Could not open file or uri... No such file or directory`

- **상태:** 인증서 요청서(CSR) 또는 서명 발급 시 특정 파일이 없다고 구동 정지되는 현상.
- **원인:** 서명을 수행하려는 현재 터미널 작업 위치(`pwd`)에 `private-key.pem` 또는 `server.csr` 같은 사전 생성 파일이 보관되어 있지 않기 때문입니다.
- **해결 방법:**

1. `ls` 명령어로 현재 필요한 원본 파일들이 해당 폴더에 살아 있는지 점검합니다.
2. 만약 루트 홈(`~`)에서 만들었다면 해당 파일을 `mv` 명령어로 실습 폴더 (`~/workspaces/day9`) 내부로 모아온 뒤 명령어를 실행해야 합니다.

▼ [Toggle] 에러 5: `invalid reference format: repository name must be lowercase` (대문자 이미지 풀링 거부) ★

- 상태: `docker pull X` 수행 시 잘못된 규격 포맷이라며 가동 실패하는 오류.
- 원인: Docker Hub 및 로컬 저장소는 주소 명세 규칙상 **이미지 이름과 레포지토리 주소에 대문자를 사용하는 것을 엄격하게 제한**합니다.
- 해결 방법: 가져오려는 타깃 이미지명을 모두 소문자로 변경하여 요청해야 합니다. (예: `docker pull ubuntu` 등)

▼ [Toggle] 에러 6: `driver failed programming external connectivity... Address already in use` (가장 자주 발생하는 포트 충돌) ★★★

- 상태: `docker run -p 8080:80 nginx` 실행 시 네트워크 구성 오류와 함께 외부 IP 바인딩에 실패했다며 기동이 차단되는 현상.
- 원인: 도커 컨테이너가 점유하려는 호스트 PC 가상머신의 `8080` 포트를 이미 호스트 본체의 다른 서비스(예: 이전에 실행해 둔 우분투의 Nginx 서비스)나 다른 실행 중인 도커 컨테이너가 먼저 선점하여 Listen 상태로 대기하고 있기 때문에 가교를 놓을 수 없는 상태입니다.
- 해결 방법 (완벽 가이드 - 점유 프로세스 모두 찾아내서 완전 종료 및 Shutdown하기):

① 1단계: 8080 포트를 무단 점유 중인 범인(PID) 추적하기

우분투 터미널에서 아래 명령어 중 하나를 사용하여 포트 `8080` 에 기생하는 주범의 PID(프로세스 번호)를 색출합니다.

```
# 방법 A: netstat 사용 (가장 고전적이고 확실한 명세)
sudo netstat -antup | grep 8080
# 출력 분석: tcp  0  0 0.0.0.0:8080  0.0.0.0:* LISTEN  917/nginx: master p... → PID가 '917'임을 판독!

# 방법 B: ss 명령어 활용 (최신 리눅스 권장)
sudo ss -tulpn | grep :8080
```

```
# 방법 C: lsof 명령어 사용 (포트 타겟 추적)
sudo lsof -i :8080
```

② 2단계: 주범 세력별 확실하게 중지/Shutdown 시키기

식별된 프로세스의 성격에 따라 아래 프로세스를 밟아 완벽하게 가용한 공간으로 반환시킵니다.

○ Case A: 호스트의 시스템 기본 서비스(우분투에 설치된 Nginx 등)인 경우

상시 실행 중인 시스템 백그라운드 데몬 서비스를 정지시킵니다. 필요시 재부팅 시에도 자동으로 구동되지 않도록 완전히 잠금(Disable) 처리합니다.

```
# 서비스 즉시 완전 중단 (Shutdown)
sudo systemctl stop nginx

# 시스템 재부팅 시에도 이 서비스가 포트를 선점하지 않도록 자동 가동 비활성화
sudo systemctl disable nginx
```

○ Case B: 다른 실행 중이거나 죽지 않은 도커 컨테이너가 점유 중인 경우

이미 가동되었던 다른 테스트용 컨테이너가 포트를 잠그고 있을 때, 해당 컨테이너 세션을 중지 및 완전 소멸시킵니다.

```
# 1. 8080 포트와 바인딩된 컨테이너 이름(또는 ID) 스캔
docker ps -a | grep 8080

# 2. 해당 도커 컨테이너 안전 정지 (Shutdown)
docker stop <컨테이너_이름_또는_ID>

# 3. 정지된 컨테이너 데이터 완전 삭제 (포트 및 네임 자원 완벽 회수)
docker rm <컨테이너_이름_또는_ID>
```

○ Case C: 정체를 알 수 없는 독립 데몬이나 고스트 프로세스인 경우

`netstat` 나 `lsof` 검색 결과로 알아낸 고유 **PID** 값을 대상으로 강제 종료 시그널(`-9`, `SIGKILL`)을 날려 완전히 프로세스를 파괴합니다.

```
# 확인한 PID가 '917'인 경우 하드 킬(Kill) 실행
sudo kill -9 917
```

③ 3단계: 포트가 완벽히 비어 있는지 대조 후 도커 가동

```
# 아무런 출력이 나오지 않는 깨끗한 공허 상태인지 최종 점검
sudo ss -tulpn | grep :8080

# 이제 충돌 없이 정상 구동!
docker run -d --name web1 -p 8080:80 nginx
```

▼ [Toggle] 에러 7: **Conflict. The container name "/web1" is already in use** (컨테이너 네임 중복 충돌) ★

- **상태:** `docker run` 명령 가동 시 이미 해당 네임(`/web1`)이 사용 중이라며 구동 거부되는 현상.
- **원인:** 이전에 실행을 시도했던 `docker run` 명령이 비록 포트 충돌 등으로 인해 정상 작동하지 않고 비정상 종료(**Exited**)되었을지라도, 지정해 준 `-name web1` 이라는 이름은 이미 메타데이터 저장소에 선점 등록되어 찌꺼기로 남았기 때문입니다.

• 해결 방법:

1. 꺼져 있거나 꼬인 비정상 컨테이너 목록까지 전부 뒤져 원인을 스캔합니다.

```
docker ps -a | grep web1
```

2. 기존에 꼬여서 상주하고 있던 좀비 컨테이너 장치를 영구 삭제한 뒤 명령을 내리거나, 다른 이름으로 컨테이너를 실행합니다.

```
docker rm web1 # 기존 꼬인 web1 컨테이너 제거
docker run -d --name web1 -p 8080:80 nginx # 재시도
```

▼ [Toggle] 에러 8: **failed to read dockerfile: open Dockerfile: no such file or directory** (도커파일 누락 에러) ★★★

- **상태:** 도커 빌드(`docker build -t mysite .`) 명령어 가동 시, 이미지를 빌드하기 위한 빌드 정의서를 읽지 못하고 컴파일이 실패하며 즉시 종료되는 현상.
- **원인:** 도커 엔진은 릴리즈 빌드(`build`) 시, 명령어를 입력한 현재 위치(`.`) 안에서 빌드 설계 도면이자 명세서 역할을 하는 **Dockerfile** (확장자 없음, 대소문자 고정) 파일을 찾습니

다. 만약 현재 작업 경로(`~/mysite`) 내부에 `index.html` 파일만 존재하고 정작 빌드 흐름을 명시한 `Dockerfile` 이 만들어져 있지 않으면 이를 열 수 없어 발생합니다.

- **해결 방법:**

1. 현재 작업 폴더(`/home/kopo20/mysite`) 경로 안에 대소문자 구분을 정확히 일치시켜 확장자 없이 `Dockerfile` 을 신규 생성합니다. (※ `dockerfile` 소문자 작성 혹은 `Dockerfile.txt` 같은 파일 형식은 인식 불가)

```
# vi 혹은 nano로 생성
vi Dockerfile
```

2. `Dockerfile` 내부에 기반이 될 Base 이미지 정보와 빌드 시 복사 및 주입할 타깃 액션을 정밀하게 작성한 뒤 저장합니다. (예: Nginx 웹 서버를 기반으로 하여 방금 만든 `index.html` 을 기본 웹 경로에 덮어씌워 빌드하는 경우)

```
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
COPY serverstatus.html /usr/share/nginx/html/serverstatus.html
COPY ChartNew/ /usr/share/nginx/html/ChartNew/
```

3. 설계 파일이 온전하게 구축되었는지 `ls` 명령어로 확인 후, 다시 도커 이미지 빌드를 실행하면 성공합니다.

```
# 파일 생성 목록 및 네임 검수
ls -l

# 이미지 빌드 재가동
docker build -t mysite .
```

▼ [Toggle] 에러 9: `bash: !: event not found` (느낌표 포함 문자열 입력 오류) ★★★★★

- **상태:** `echo "<h1>Realtime Change Without Rebuild! Magic!</h1>" > ~/mysite/index.html` 입력 시 쉘 구동 오류 발생.
- **원인:** 리눅스 배시 쉘(Bash Shell) 환경에서 큰따옴표(`"..."`) 기호 내에 느낌표(`!`)가 들어가면, 쉘은 이를 일반 문자열이 아니라 **이전 실행 이력 목록을 호출 및 재실행하려는 역사 확장(History Expansion/Event Designator)** 단축키로 해석하려 시도합니다. 이전에

수행한 명령 이력 중 'Magic' 등으로 시작하는 키워드가 존재하지 않으므로 구동 실패 에러가 떨어지게 됩니다.

- **해결 방법:**

1. 배시의 역사 단축키 확장이 차단되는 작은따옴표('...')로 문자열을 감싸서 명령을 재시도합니다 (가장 권장하는 정석 해결법):

```
echo '<h1>Realtime Change Without Rebuild! Magic!</h1>' > ~/mysite/index.html
```

2. 큰따옴표를 꼭 써야 하는 스크립트라면, 느낌표 기호 바로 앞에 역슬래시(\) 기호를 붙여 셸의 특수 기능을 무력화(Escape) 시킵니다:

```
echo "<h1>Realtime Change Without Rebuild! Magic\!</h1>" > ~/mysite/index.html
```